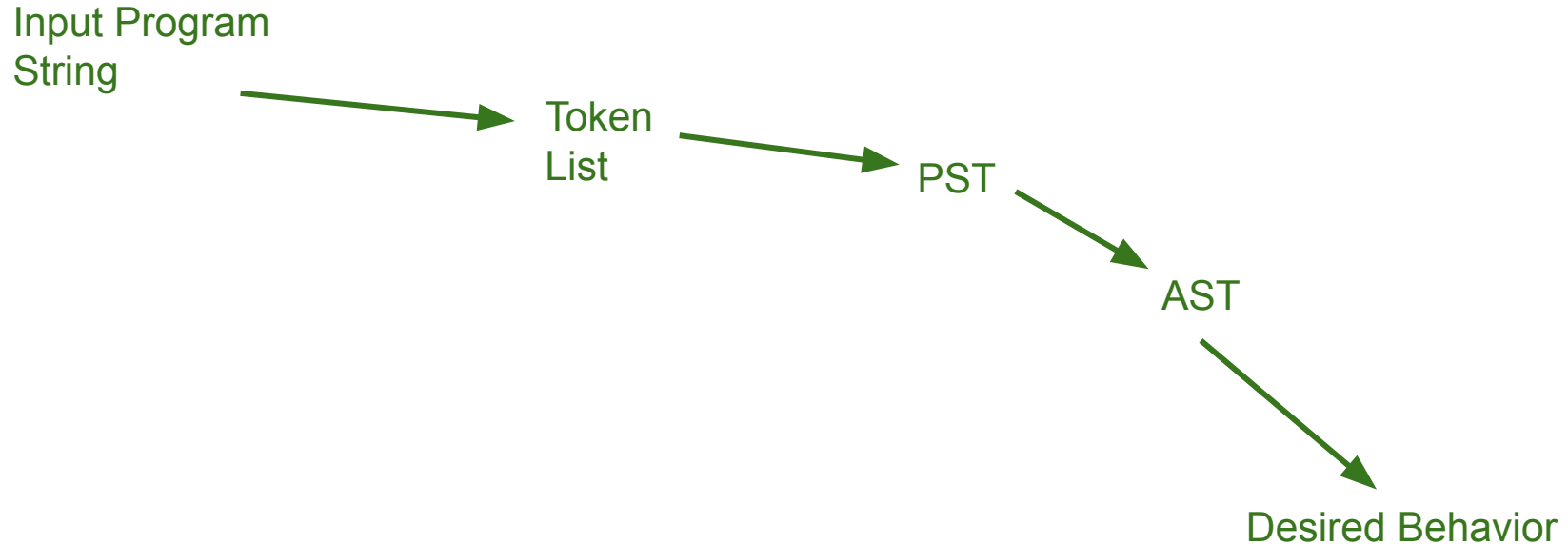


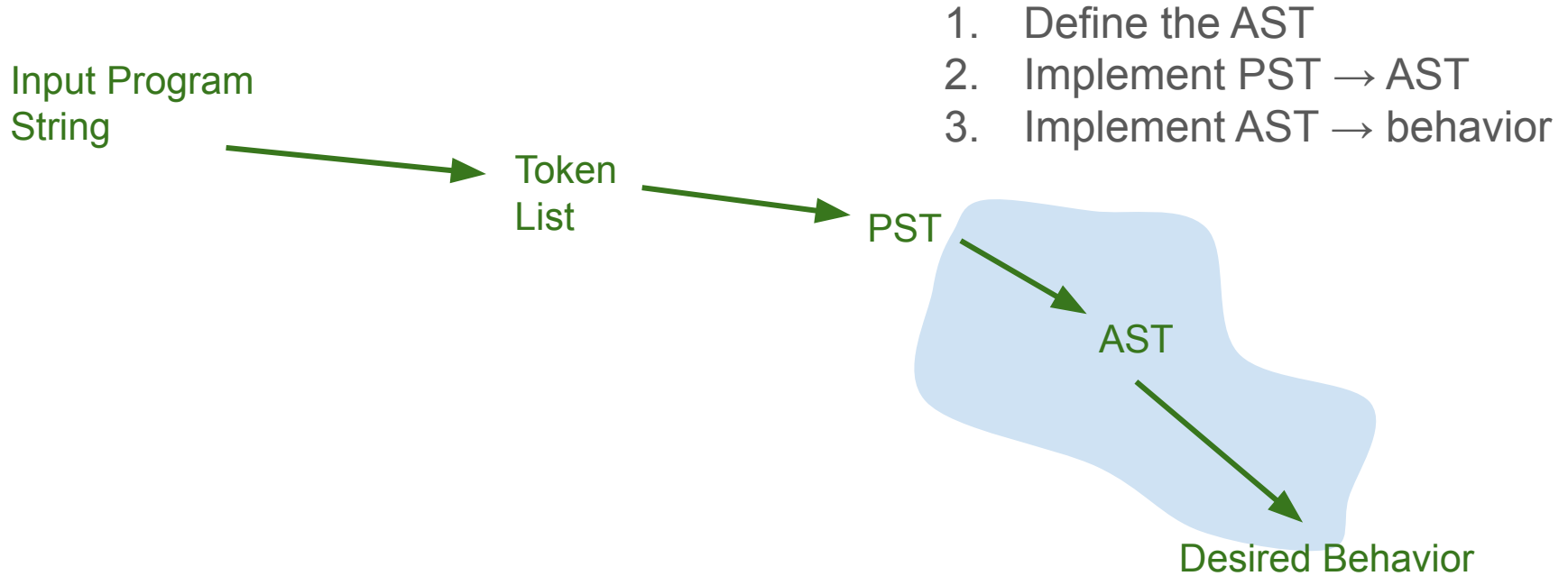
Lecture 15

Modules

How to Add a Language Feature



How to Add a Language Feature



Group Work

Implement unary negation, $\sim$, in our prefix arithmetic language.

```
type expr =  
  ...  
  | <1. YOUR NEW CASE>  
  
let rec ast_of_pst (pst : pst) : expr =  
  match pst with  
  ...  
  | <2. YOUR NEW CASE>  
  
let rec interpret (ast : expr) : int =  
  match ast with  
  ...  
  | <3. YOUR NEW CASE>
```

```
type expr =  
  | Int of int  
  | Add of expr * expr  
  | Sub of expr * expr  
  | Mul of expr * expr  
  
let rec interpret (ast : expr) : int =  
  match ast with  
  | Int i -> i  
  | Add (l, r) -> interpret l + interpret r  
  | Sub (l, r) -> interpret l - interpret r  
  | Mul (l, r) -> interpret l * interpret r  
  
let rec ast_of_pst (pst : pst) : expr =  
  match pst with  
  | Atom s -> Int (int_of_string s)  
  | Node [ Atom "+"; p1; p2 ] -> Add (ast_of_pst p1, ast_of_pst p2)  
  | Node [ Atom "-"; p1; p2 ] -> Sub (ast_of_pst p1, ast_of_pst p2)  
  | Node [ Atom "*"; p1; p2 ] -> Mul (ast_of_pst p1, ast_of_pst p2)  
  | _ ->  
    failwith  
      "this pst was not parseable as an AST for infix arithmetic"
```

Modules

For larger programs, one “top-level” sequence of bindings is poor

- A binding can use *all* earlier (non-shadowed) bindings

So OCaml has *structures* to define *modules*

```
module MyModule = struct bindings end
```

- Inside a module, can use earlier bindings as usual
 - Can have any kind of binding (let, type, exception, ...)
- Outside a module, refer to earlier modules' bindings via

`ModuleName.bindingName`

- Like `List.fold_left`, etc. now can define your own modules

Example

```
module MyMathLib = struct
  let rec fact x =
    if x = 0
    then 1
    else x * fact (x - 1)
  let half_pi = Float.pi /. 2.0
  let doubler x = x *. 2
end
```

Implicit modules via files

OCaml also has a built-in convenience:

- A file `foo.ml` has an *implicit*

```
module Foo = struct ... end
```

“around it”

- So different files do not shadow, but need to access bindings from other files with **Foo.bar** syntax

Optional: Open

`open` `ModuleName` to get “direct” access to a module’s bindings

- Never necessary; just a convenience; often bad style
- Often better to create local val-bindings for just the bindings you use a lot, e.g., `let map = List.map`
 - But doesn’t work for patterns
 - And `open` can be useful, e.g., for testing code

Namespace management

So far, this is just namespace management

Giving a hierarchy to names to avoid shadowing

- Yes, can have modules inside modules
- Allows different modules to reuse names, e.g., **map**
- Very important, but not very interesting

Module Types (aka Signatures)

- A *signature* is a type for a module
 - What bindings does it have and what are their types
- Can define a signature and *ascribe* it to modules – example:

```
module type MATHLIB = sig
  val fact : int -> int
  val half_pi : float
  val doubler : int -> int
end
module MyMathLib : MATHLIB = struct
  let rec fact x = ...
  let half_pi = Float.pi /. 2.0
  let doubler x = x *. 2
end
```

In general

- Signatures

```
module type SIGNAME = sig  
    types-for-bindings  
end
```

- Can include variables, types, and exceptions defined in module

- Ascribing a signature to a module

```
module ModuleName : SIGNAME = struct  
    bindings  
end
```

- Module *will not type-check* unless it *matches the signature*, meaning it has all the bindings at the right types

Hiding things

Real value of signatures is to to *hide* bindings and type definitions

- So far, just documenting and checking the types

Hiding implementation details is the most important strategy for writing correct, robust, reusable software

So first remind ourselves that functions already do well for some forms of hiding...

Hiding with functions

These three functions are totally equivalent: no client can tell which we are using (so we can change our choice later):

```
let double x = x*2
let double x = x+x
let y = 2
let double x = x*y
```

Defining helper functions locally is also powerful

- Can change/remove functions later and know it affects no other code

Would be good to have “private” top-level functions too

- So two functions could share a helper function
- OCaml does this via signatures that omit bindings...

OCaml pragmatics

- Recall file `foo.ml` implicitly defines the `Foo` module with no explicit `module Foo = struct ... end`
- Similarly, if `foo.mli` is present, it is the signature for `Foo` with no explicit `module type SOMENAME = sig ... end`
 - `foo.ml` is type-checked as though it is `module Foo : SOMENAME = ...`
 - Other modules type-checked with `foo.mli` hiding things
- If no `foo.mli` present, then just “nothing hidden”

Module + Signature Example: Positives

```
module type POSITIVE = sig
  type t
  val of_int : int -> t option
  val add : t -> t -> t
  val mul : t -> t -> t
  val sub : t -> t -> t option
end
```

```
module Positive2 : POSITIVE2 = struct
  type t = int
  let of_int x =
    if x > 0 then
      Some x
    else
      None
  let add a b = a + b
  let mul a b = a * b
  let sub a b = of_int (a - b)
end
```

Group Work

Fill in the implementations of **Ne1A**, **Ne1B**, and **Ne1C** to adhere to the **NONEMPTYLIST** signature.

```
module type NONEMPTYLIST = sig
  type 'a t
  val single : 'a -> 'a t
  val cons : 'a -> 'a t -> 'a t
  val tl : 'a t -> 'a t (* can fail *)
  val hd : 'a t -> 'a (* never fail *)
  val len : 'a t -> int
end
```

```
module Ne1A : NONEMPTYLIST = struct
  type 'a t = 'a * 'a list
  let single x = (x, [])
  let cons x1 (x2, xs) = (x1, x2 :: xs)
  ...
end
```

```
module Ne1B : NONEMPTYLIST = struct
  type 'a t = 'a list
  ...
end
```

```
module Ne1C : NONEMPTYLIST = struct
  type 'a t =
    | Single of 'a
    | Cons of 'a * 'a t
  ...
end
```